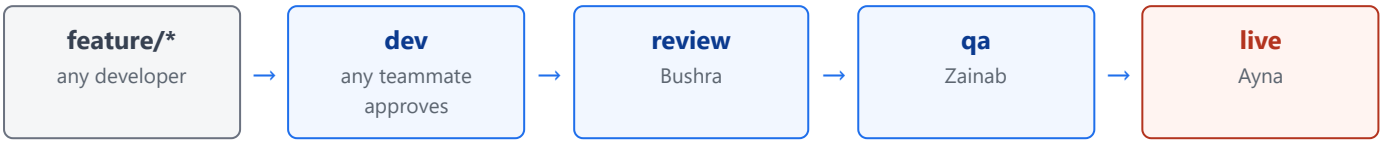
Government Procurement — How We Ship

Team workflow guide · Repository `Sohaib-Ahmed869/government-procurement` · Jira project **GOV**

The promotion chain

Code moves in one direction only. Every arrow is a Pull Request that must be approved by that stage's gatekeeper. Nothing skips a stage.



Who guards what

Branch	Gatekeeper	Purpose	What it takes to merge in
`dev`	None specifically — any teammate	Integration	1 approval + CI green
`review`	**Bushra** (@Bushra123sajid)	Code review	1 approval **from Bushra** + CI green
`qa`	**Zainab** (@ZainabFatima10)	Testing	1 approval **from Zainab** + CI green
`live`	**Ayna** (@AynaSulaiman04)	Production	1 approval **from Ayna** + CI green, **then** a second deploy approval

The rules that actually matter

1. **Branch names carry the ticket key, in capitals.** `feature/GOV-123-add-tender-form`. Lowercase `gov-123` will not link to Jira, and Jira gives no warning — the ticket simply never moves.
2. **Commit messages start with the key too:** `GOV-123: add tender form validation`.
3. **Branch off `dev`, never off `live`.** `dev` is the default branch, so a fresh clone already puts you there.
4. **Nobody approves their own PR.** GitHub blocks it. Every change is seen by a second person.
5. **Force-pushing to the four long-lived branches is blocked.** History on `dev`, `review`, `qa`, `live` cannot be rewritten.
6. **CI must pass before any merge.** It runs on every PR into all four branches.

> **Promotion PRs will show one conflict — this is expected.** Each branch has a different `.github/CODEOWNERS` naming its own gatekeeper, so a `dev → review` or `qa → live` PR always conflicts on that single line. **Always resolve by keeping the DESTINATION branch's version.** If you keep the source version, you hand that branch's gate to the wrong person.

What each person does

Find your name. These are the only steps you need.

Developer — writing a feature

Sohaib & any developer

1. Pick your Jira ticket, e.g. **GOV-123**. Note the key exactly.
2. Start from the latest `dev`:
   `git checkout dev && git pull`
   `git checkout -b feature/GOV-123-short-desc`
   → Jira moves the ticket to **In Progress** automatically.
3. Commit as you work: `git commit -m "GOV-123: what changed"`
4. Push: `git push -u origin feature/GOV-123-short-desc`
5. Open a PR **into** `dev` on GitHub.
   → Jira moves the ticket to **In Review**.
6. Wait for CI to go green, then ask a teammate for the 1 required approval.
7. Merge. Your feature is now on `dev`. Delete the feature branch.

You do not promote your own work past `dev`. Promotion happens in batches — see below.

Bushra — code review gate

review

1. When work on `dev` is ready to advance, a `dev → review` PR is opened.
2. GitHub requests your review automatically — you own every file on `review`.
3. Read the diff. Check code quality, naming, structure, obvious bugs.
4. Resolve the `.github/CODEOWNERS` conflict by **keeping `review`'s version** (`* @Bushra123sajid`).
5. **Approve** to let it through, or **Request changes** to send it back. It cannot merge without you.

Zainab — QA gate

qa

1. A `review → qa` PR is opened once code review has passed.
2. GitHub requests your review automatically.
3. Test the actual behaviour — not the code. Does the feature do what the ticket says?
4. Resolve the CODEOWNERS conflict by **keeping `qa`'s version** (`* @ZainabFatima10`).
5. **Approve** when it behaves correctly.
   → On merge, Jira moves the ticket to **In QA**.

Ayna — production gate

live

You approve twice. These are two separate gates and both are required.

1. **Gate 1 — the PR.** A `qa → live` PR is opened. GitHub requests your review; you own every file on `live`. Resolve the CODEOWNERS conflict by **keeping `live`'s version** (`* @AynaSulaiman04`), then **Approve** and merge.
   → Jira moves the ticket to **Done**.
2. **Gate 2 — the deployment.** Merging starts the Deploy workflow, which **pauses** and waits for you. Go to the repo's **Actions** tab, open the waiting **Deploy** run, and click **Review deployments → live → Approve and deploy**. Nothing reaches production until you click this.

You will also get an email for the pending deployment — approving from there works too.

How Jira tracks the ticket

Status changes are automatic. Nobody drags cards on the board.

The ticket's journey

Status	What causes it	Who triggers it
To Do	Ticket created and waiting	Whoever plans the work
In Progress	A branch named `feature/GOV-123-…` is created	Developer
In Review	A PR into `dev` is opened	Developer
In QA	A PR into `qa` is merged	Zainab
Done	A PR into `live` is merged	Ayna

> **The one thing that breaks this:** the ticket key must appear in the branch name and commit messages in **capitals** — `GOV-123`, not `gov-123`. Jira matches on the exact key. If it doesn't match, the automation still reports success — it just silently moves nothing. If a ticket isn't moving, this is almost always why.

Note on "Ready for Release": this status exists on the board but has no automation attached, so nothing moves into it by itself. Tickets go straight from **In QA** to **Done**. Move it by hand if you want to use it.

Checking your work is linked

Open the Jira ticket and look at the **Development** panel on the right. You should see your branch, commits, and pull request listed there. If that panel is empty, the key wasn't matched — rename the branch or add the key to a new commit message.

What runs automatically

Workflow	When	What it does
CI	Every PR into the four branches	Installs dependencies and runs the test suite. Must pass before merge.
Deploy	Every push to the four branches	Deploys to the matching environment. `live` waits for Ayna's approval.
Nightly	02:00 UTC daily	Runs the full test suite against `dev`, catching problems overnight.

Quick reference

Task	Command
Start a ticket	`git checkout dev && git pull` `git checkout -b feature/GOV-123-short-desc`
Save work	`git commit -m "GOV-123: description"`
Publish branch	`git push -u origin feature/GOV-123-short-desc`
Get latest `dev` into your branch	`git fetch origin && git merge origin/dev`

> **Stuck?** If a PR won't merge, check three things in order: (1) is CI green, (2) has the right gatekeeper approved, (3) is the CODEOWNERS conflict resolved to the destination branch's version. It is nearly always one of these.